Single-store e-commerce blueprint

How to build one complete e-commerce website — storefront, backend API, database, admin dashboard, checkout, payments, shipping, WhatsApp and tracking — the way this repository does it. It is written for a developer starting the next project: what to build, in which order, which tables to create and which mistakes we already made so you don't.

The reference implementation is the **Morbix** store (`Morbix/`) on the shared **Backend** (`Backend/`) and **Dashboard** (`Dashboard/`). This repository serves several stores from one backend (every table carries `brand_id`); for a single store keep the `brand_id` column anyway — one row in `brands` — so the project can grow into a second store without a migration.

Provider names are kept generic: **payment gateway**, **shipping provider**, **hosted checkout**, **WhatsApp Business API**. Where a file name in this repo names the provider we used, it is given only so you can find the code.

---

1. The shape of the system

```
Shopper's phone
  │
  ▼
Storefront (Next.js on Vercel) ──── edge middleware: visit counter, Meta/GA cookies
  │  HTTPS + X-Brand-Name header
  ▼
Backend API (Node + Express on cPanel/any Node host) ── cron jobs, webhooks
  │                          │
  ▼                          ├── Payment gateway (orders, webhooks, refunds)
MySQL                        ├── Shipping provider (rates, booking, tracking webhooks)
                             ├── Hosted checkout (optional one-tap checkout)
                             ├── WhatsApp Business API (OTP, order updates, chat)
                             ├── Meta Conversions API + Google Analytics Measurement Protocol
                             └── Telegram (staff alerts)
Admin Dashboard (Next.js on Vercel) ── same API, admin routes (separate admin host)
```

Part	Stack	Where in this repo	Hosting
Storefront	Next.js 15 (App Router), React 18, plain CSS	`Morbix/`	Vercel
Backend API	Node 20+, Express 5, Sequelize 6, MySQL 8	`Backend/`	cPanel Node app (Passenger) via FTP deploy; any Node host works
Admin Dashboard	Next.js, React Query, Tiptap editor	`Dashboard/`	Vercel
Shared storefront code	edge middleware, pixel helpers	`shared/frontend/` (copied into each store by `scripts/sync-frontend-shared.mjs`)	—
Deploys	GitHub Actions	`.github/workflows/`	push to `main` deploys everything

Rule of thumb: the storefront never talks to a third party with a secret. Every secret lives in the backend (env or encrypted `brand_settings`), and the storefront only calls our API.

---

2. Build order (phases)

Build in this order. Each phase ends with something a shopper or the owner can use.

#	Phase	Done when
1	**Backend skeleton** — Express app, MySQL, Sequelize models, auth, error handling, health route	`GET /health` answers; tables create on boot
2	**Catalogue** — categories, products, variations (size/colour), images, attributes, stock	Admin can add a product with variants and photos; API lists it
3	**Storefront browse** — home, collections, product page, search, SEO	A shopper can find and view a product; pages are static/ISR and fast
4	**Cart + account** — cart, wishlist, WhatsApp OTP login, addresses	A shopper can log in with their phone and keep a cart
5	**Checkout + payments** — order creation, coupons, shipping fee, COD + online payment, payment webhook	A real order is placed and paid; stock is reserved then deducted
6	**Admin Dashboard** — orders, products, customers, coupons, settings	Owner runs the store without touching the database
7	**Shipping** — serviceability by pincode, booking, labels, tracking webhooks, NDR, returns/refunds	An order goes from “placed” to “delivered” (or RTO) on its own
8	**WhatsApp** — order confirmations, shipping updates, abandoned cart, inbox	Customers get updates; staff reply from the Dashboard
9	**Tracking** — visit counter, funnel events, Meta pixel + Conversions API, GA4 + Measurement Protocol, traffic report	Ads Manager and our report agree within ~10%
10	**Growth** — reviews, loyalty, bundles/pack pricing, back-in-stock/price-drop alerts, blog, catalogue feeds	—
11	**Operations** — cron jobs, reconciliation, alerts, backups, privacy (data export/erasure)	Nothing needs a person to remember it

---

3. Database

MySQL 8, InnoDB, `utf8mb4`. Sequelize models live in `Backend/model/` (one file per table, relations in `model/associations.js`). Money is `DECIMAL(12,2)`, never float. Every business table has `brand_id`, `created_at`, `updated_at`.

3.1 Core tables (build these first)

Table	Purpose	Key columns
`brands`	The store itself (one row)	`name`, `slug`, `domain`, `logo_url`, `primary_color`, `status`
`brand_settings`	Every configurable key per store — payment, shipping, WhatsApp, tracking IDs, policies	`brand_id`, `key`, `value`, `is_encrypted`, `category`
`users`	Customers and staff	`phone`, `email`, `password` (staff only), `role` / `roles`, `loyalty_points`, `deleted_at`
`guest_users`	Checkout without an account	`phone`, `email`
`shipping_addresses`	Saved addresses	`user_id` / `guest_user_id`, `full_name`, `address`, `city`, `state`, `pincode`, `phone`, `is_default`, `address_hash`
`categories`	Collections	`name`, `slug`, `image`, `status`
`products`	A product	`name`, `slug`, `description`, `status`, `badge`, `avg_rating`, `review_count`, `total_sold`, `weight`, `gst_rate`
`product_variations`	What is actually sold (size × colour)	`product_id`, `sku`, `price`, `cost_price`, `stock`, `attributes` (JSON), `status`
`product_images`	Photos, optionally per variation	`product_id`, `product_variation_id`, `image_url`, `display_order`, `is_primary`
`attributes`, `attribute_values`	Size, colour… and their values	`name`, `type` / `value`
`product_seo`, `seo_metadata`	Titles, descriptions, canonical, JSON-LD per product and per page	`meta_title`, `meta_description`, `canonical_url`, `structured_data`
`carts`, `cart_items`	Server cart (logged-in shoppers)	`user_id`, `status` / `variation`, `quantity`, `price`
`wishlists`	Saved products	`user_id`, `product_id`
`coupons`, `coupon_usages`	Discounts	`code`, `type` (percent/flat), `value`, limits, dates
`shipping_fees`	Flat/threshold shipping fee	`fee`, `brand_id`
`orders`	One order	`order_number`, `user_id` / `guest_user_id`, `total_amount`, `discount_amount`, `shipping_fee`, `final_amount`, `payment_type` (cod/online), `payment_status`, `status`, `coupon_id`, `utm_tracking_id`, shipping/tracking columns

Table	Purpose	Key columns
`order_items`	Lines of an order	`order_id`, `product_id`, `variation_id`, `quantity`, `price`, `discount`, `subtotal`
`order_status_history`	Every status change, who and why	`order_id`, `status`, `notes`, `updated_by`
`payments`	Gateway payments and their state	`order_id`, `transaction_id`, `gateway order id`, `amount_paid`, `status`
`order_shipments`	Courier booking per order	`provider`, `waybill`, `tracking_url`, `label_url`, `sync_status`

3.2 Growth and operations tables

Table	Purpose
`returns`	Return/refund requests: items, reason, photos, resolution, UPI/phone for COD refunds, pickup AWB, status
`reviews`, `review_images`	Reviews with verified-purchase flag and moderation
`bundle_tiers`	Pack pricing (“2 pairs ₹X, 3 pairs ₹Y”) per product
`loyalty_transactions`	Points earned/spent/expired with running balance
`abandoned_carts`	Phone + items for WhatsApp recovery
`lead_captures`	Popup phone captures with coupon
`push_subscriptions`	Web push endpoints
`whatsapp_conversations` (+ messages, quick replies)	The WhatsApp inbox
`contact_messages`	Contact form
`faqs`, `policies`, `sliders`, `blog_*`	Content managed from the Dashboard
`address_quality_scores`	Delivery success/failure per address hash (blocks risky COD)
`consent_logs`	Cookie/marketing consent with notice version (privacy law)

3.3 Tracking tables (the traffic report)

Table	Written by	What it holds
`utm_tracking`	storefront page script (`POST /api/i/s`), once per tab session	One row per **tracked visit**: `session_id`, `utm_*`, `landing_page`, `referrer`, `ip_address`, `user_agent`. Orders link to it by `orders.utm_tracking_id`
`funnel_events`	storefront (`POST /api/i/e`)	`view_item`, `add_to_cart`, `begin_checkout`, … per `session_id`
`visit_hits`	store edge middleware (`POST /api/i/v`)	Daily totals of page loads/sessions by channel, campaign, region, device — counted before any JavaScript runs

Table	Written by	What it holds
`page_hits`	storefront (`POST /api/i/pv`)	Cookieless daily page counts per channel
`meta_order_tracking`	backend, when Purchase is sent	Browser ids of each order's buyer, to send Delivered/Cancelled/Returned later
`ad_spend_daily`	cron (Meta Marketing API)	Spend, impressions, link clicks per ad per day, for ROAS

---

4. Backend API

`Backend/index.js` builds the app: `helmet` , `compression` , CORS for the store domains, cookies, rate limits, CSRF for cookie-auth routes, JSON body limits, then `routes/routesManager.js` mounts everything under `/api` .

4.1 Folder layout

Folder	Holds
`config/`	DB connection, env loading, CORS, logging, cron jobs, OpenAPI
`model/`	Sequelize models + associations
`routes/`	Express routers (thin)
`controller/`	Request handling: validate, call a service, shape the response
`services/`	Business logic: order creation, stock, payments, shipping, WhatsApp, reports
`integration/`	Meta pixel / Conversions API, Google Analytics
`middleware/`	Auth (customer + admin), store resolution (`X-Brand-Name`), CSRF, uploads, webhook signatures, validation
`utils/`	Small helpers: encryption, bot detection, traffic channel rules, dates (India time), image resizing
`tests/`	Jest smoke/integration tests — the deploy runs them first

4.2 Route groups

Prefix	For
`/api/auth` , `/api/users`	WhatsApp OTP login, profile, staff login (JWT)
`/api/products` , `/api/categories` , `/api/attributes` , `/api/search`	Catalogue (public reads, admin writes)
`/api/cart` , `/api/wishlist` , `/api/shipping-addresses` , `/api/shipping-fees`	Shopper state
`/api/checkout` , `/api/orders` , `/api/payments` , `/api/coupons`	Placing and paying orders

Prefix	For
`/api/custom-checkout`	Hosted one-tap checkout: token, catalogue pull, order webhook
`/api/returns`, `/api/ndr`, `/api/pickup-schedule`	After-sale
`/api/reviews`, `/api/loyalty`, `/api/leads`, `/api/notifications`	Growth
`/api/sliders`, `/api/blogs`, `/api/policies`, `/api/seo`, `/api/public/*`	Content and public config (tracking IDs, organisation schema)
`/api/whatsapp`	Webhook + inbox
`/api/i/s`, `/api/i/e`, `/api/i/v`, `/api/i/pv`	Tracking pings (short paths so ad blockers don't match them)
`/api/facebook-pixel`, `/api/google-analytics`	Server copies of events, admin "Check" buttons
`/api/reports`, `/api/dashboard`, `/api/admin/*`	Dashboard only (admin auth)

4.3 Rules that saved us

- **Store from a header, never a default.** Every storefront request sends `X-Brand-Name`; `middleware/brandMiddleware.js` resolves it. A public endpoint must refuse an unknown store instead of falling back to store #1 — otherwise one store's events land in another store's pixel.
- **Secrets in `brand_settings`, encrypted** (`utils/encryption.js`, key in `DATA_ENCRYPTION_KEY`). The Dashboard edits them; no redeploy to change a key.
- **Order creation is one transaction** (`services/orderCreationService.js`): validate cart → price from the database (never trust the browser's price) → coupon → shipping fee → create order + items → reserve stock → commit. Side effects (WhatsApp, pixel, emails) run **after commit** (`transaction.afterCommit`), never inside it.
- **Webhooks are verified and idempotent.** Check the signature (`middleware/webhookSignature.js`), then make the handler safe to run twice (unique keys, "only if status changes").
- **Money in paise or DECIMAL**, rounding once at the end.
- **India time everywhere for reports** (`utils/dateRange.js`, `APP_TZ_OFFSET=+05:30`): a day is 00:00–23:59 IST, not UTC.
- **Two copies of the API in production:** the main app (runs cron jobs) and an admin copy (`APP_ROLE=admin`, separate host) so heavy Dashboard reports never slow the shop.

---

5. Storefront

Next.js App Router. Reference: `Morbix/`.

5.1 Pages

Route	Rendering	Notes
`/`	Static, revalidate 60 s	Hero from `sliders`, featured products

Route	Rendering	Notes
`/collections`, `/collections/[handle]`	Server	Filters, pagination
`/products/[slug]`	**Static + ISR** (built per product)	The money page — must be fast. Colour deep links via `/products/[slug]/c/[color]`
`/search`	Server	Uses `/api/search`
`/account/*`, `/login`, `/register`	Client	OTP login, orders, returns, settings
`/order-confirmation`	Server + client	Fires Purchase (pixel + GA) with the order's event id
`/track-order`, `/wishlist`, `/contact`, `/about`, `/journal/*`, `/policies/[name]`	Static/ISR	—
`/sitemap.xml`, `/robots.txt`, `/facebook-catalog.xml`, `/google-merchant.xml`	Generated	Product feeds for Meta and Google catalogues

5.2 Folder layout

Folder	Holds
`app/`	Routes and the root layout (fonts, metadata, Analytics, VisitTracker)
`components/`	`layout/` (header, footer, Analytics), `product/`, `cart/`, `checkout/`, `account/`, `home/`, `ui/`
`context/`	`AuthContext`, `CartContext`
`lib/api/`	One small module per API area (`client.js` adds `X-Brand-Name` and handles errors)
`utils/pixel.js`	Meta pixel + GA4 helpers shared by all stores (from `shared/frontend/`)
`middleware.js`	Edge: visit counter + first-party Meta/GA cookies (from `shared/frontend/visitCounter.js`)

5.3 Performance rules (measured, not guessed)

- Product pages **static with ISR**; fetch product data at build/revalidate time, not in the browser.
- **Right-size images** (the API serves resized copies via `utils/imageResizer.js` + `sharp`); give the main product photo `priority` and real `sizes`.
- **Third-party scripts late**: Meta pixel and GA4 load after the page is usable — except for visitors who arrive from an ad (`fbclid` / `gclid` / `utm_source` in the address), who get them at once so short visits still count.
- Read tracking IDs **on the server in the layout** (cached 5 min) and pass them down, so analytics doesn't wait for an extra API round trip.
- One font family via `next/font`; no layout shift from late fonts or images.
- Preconnect to the API host in `<head>`.

---

6. Checkout, payments and orders

6.1 Two ways to check out

1. **Own checkout** — cart → address → shipping fee/serviceability → coupon → COD or online payment → payment gateway order → gateway popup → verify signature on the server → order confirmed.
2. **Hosted one-tap checkout** (what Morbix uses) — the store asks the backend for a checkout token (`/api/custom-checkout`), opens the provider's checkout, and the provider calls our webhook with the order. The backend re-fetches the order from the provider (never trusts the webhook body alone), stamps the store and the visit on it and creates the local order. Products are pushed to the provider only when price/title/stock status change (remembered in `custom_checkout_pushes`) so the button opens fast.

6.2 Order life cycle

`pending` → `awaiting_confirmation` (COD to confirm) → `confirmed` → `processing` → `booked` → `pickup initiated` / `manifested` → `in transit` / `shipped` → `out for delivery` → `delivered` Side branches: `cancelled`, `undelivered` (NDR), `rto` / `rto delivered` / `returned_rto` (returned to origin), `return_initiated`, `exception`. Payment status is separate: `pending`, `paid`, `failed`, `refund_pending`, `partial_refund`, `refunded`, `cancelled`.

Every change goes through the `Order` model so one hook (`services/metaOrderOutcomes.js`, `services/orderEvents.js`) can send WhatsApp updates, Meta OrderDelivered/Cancelled/Returned and GA refunds.

6.3 Payments

- Create the gateway order on the server with the **database total**.
- Verify the payment signature on the server; also handle the gateway's **webhook** (the shopper may close the tab).
- **Reconciliation cron** re-checks unpaid online orders; **cleanup cron** cancels abandoned unpaid ones and releases stock.
- Refunds from the Dashboard; listen to refund processed/failed webhooks.
- COD: block risky addresses using `address_quality_scores`; confirm COD orders on WhatsApp before booking the courier.

6.4 Stock

Reserve on order creation (`services/stockReservationService.js`), deduct on confirmation, release on cancel/expiry. A cron clears stuck reservations.

---

7. Shipping, returns and NDR

- **Serviceability**: pincode → can deliver? COD allowed? delivery estimate (`utils/serviceability.js`).
- **Booking**: batch cron books confirmed orders with the shipping provider, stores AWB, label and tracking URL in `order_shipments`.

- **Tracking**: provider webhook + a status refresh cron update the order status.
- **NDR (failed delivery)**: WhatsApp the customer for a new date/address and pass it back to the provider.
- **Returns**: request within 7 days with photos → admin approves → pickup → refund (online: gateway refund; COD: UPI/phone payout with proof).

---

8. WhatsApp

- **Login OTP** over WhatsApp.
- **Order messages**: placed, COD confirmation, shipped, out for delivery, delivered, review request.
- **Recovery**: abandoned cart and checkout drop-off messages with a cart link (`/api/cart-recover/...` reopens the cart with a coupon).
- **Inbox** in the Dashboard with assignment, notes, quick replies and an optional AI reply assistant.
- Use approved **templates** for anything outside the 24-hour window.

---

9. Tracking and the traffic report

This is where we spent the most time. Build it right from day one.

9.1 What to install

Tool	Browser	Server copy	Dedupe
Meta pixel	PageView, ViewContent, AddToCart, InitiateCheckout, Purchase	Conversions API from the backend for the same events, plus OrderDelivered / OrderCancelled / OrderReturned / CheckoutAbandoned	Same `event_id` in both (`Purchase_<order_number>`)
Google Analytics 4	page_view, view_item, add_to_cart, begin_checkout, purchase	Measurement Protocol `purchase` + `refund` , with the browser's `client_id` and `session_id`	Same `transaction_id`
Own visit tracker	One visit per tab session + funnel events	—	Session id
Edge counter	Every real page load (before JavaScript)	—	—

Match quality for Meta: first-party `_fbp` / `_fbc` cookies set **at the edge** (`shared/frontend/visitCounter.js`), a hashed visitor id (`_mvid`) as `external_id` , hashed phone/email of returning buyers, IP and user agent on server events.

9.2 Counting visits correctly

- **Bots out**: user-agent list (`utils/botDetect.js`) including Meta's crawlers, plus **Meta's own network (AS32934)** — Meta's ad review opens landing pages with Facebook-app user agents.

- **Previews out**: skip link preloads (`Purpose` / `Sec-Purpose` / `X-Purpose: preview|prefetch`) at the edge, and only record a visit once the page is actually on screen (`document.visibilityState` , `prerendering`).
- **Source rules** (`utils/channel.js`): UTM tags → known referrer → ad click ids (`gclid` = Google Ads, `fbclid` = Meta, Instagram when the in-app browser is Instagram) → in-app browser user agent → Direct.
- **Fresh start**: when tracking changes, set a “data from” moment (`TRAFFIC_DATA_FROM` in `reportsController.js`) so old rows don’t mix in.

9.3 Why three tools never show the same number

Number	Counts	Typical
Meta **link clicks**	Taps on the ad	Highest
Meta **landing page views**	Taps where the pixel loaded	75–90% of link clicks
Our tracked visits	Pages that opened and ran our script	Within ~5–10% of landing page views
GA4 sessions	Pages where gtag.js loaded and sent a hit	Lower again (late load, blockers)

Compare **landing page views ↔ our visits**. If visits are far below link clicks, the page is too slow on mobile data or the placement brings accidental taps (Audience Network).

9.4 Ads checklist

- Every ad carries URL parameters: `utm_source=facebook&utm_medium=paid_social&utm_campaign={{campaign.name}}&utm_term={{adset.name}}&utm_content={{ad.name}}&placement={{placement}}`
- `placement` tells the report where on Meta each visit came from. Placements like **WhatsApp Status** and **Audience Network** open the link in the phone’s normal browser (Chrome), with no Facebook-app label and no referrer, so without it they look like plain Facebook or Direct visits.
- Without them GA files Meta ad visits under **(direct) / (none)** — GA ignores `fbclid` , and Android often opens the link without a referrer.
- Campaign goal **Sales** (Purchase), not Traffic, once the pixel fires Purchase.
- Domain verified in Meta Business (meta tag in the root layout).
- Remove the Conversions API **test event code** after testing.
- Connect the ad account in the Dashboard **Ads** tab so spend sits next to orders (ROAS).

---

10. Admin Dashboard

Next.js + React Query, talking to the admin API host. Modules we needed, in the order owners use them:

1. **Home (today)** — orders, revenue, pending actions.
2. **Orders** — list, filters, detail, status, courier booking, labels, invoices, refunds.
3. **Products** — products, variants, images, attributes, pack pricing, SEO badge.
4. **Customers** — profile, orders, loyalty, WhatsApp history; data export/erasure.
5. **Returns / NDR / Exceptions** — the problem queue.
6. **Coupons, Loyalty, Reviews, Leads.**

7. **WhatsApp inbox + broadcasts.**
8. **Content** — sliders, blog, FAQs, policies, SEO per page.
9. **Reports** — Overview, Revenue & Profit, Traffic (with Visit check), Ads (spend vs orders), Products, Locations, Coupons, Customers, Inventory. All dates in India time; CSV export; Refresh button.
10. **Settings (per store)** — payment, shipping, WhatsApp, tracking IDs (with **Check Meta pixel** and **Check Google Analytics** buttons), policies. Secrets stored encrypted.
11. **System** — job runs, server health, settings audit, staff activity.

---

11. Scheduled jobs

Defined in `Backend/config/cronJobs.js`; run only on the main app (not the admin copy). Each run is logged (`services/jobRunService.js`) so the Dashboard shows failures.

Job	Why
Courier status refresh / booking batch	Orders move without staff
Abandoned cart + checkout drop-off messages	Recover sales
Payment reconciliation, unpaid order cleanup, stuck stock-hold cleanup	Money and stock stay correct
Back in stock / price drop, win-back, review requests, after-delivery suggestions	Repeat sales
Loyalty points expiry, product badges refresh	Keep rules true
Meta ad spend sync	ROAS in the Ads report
Old data cleanup (privacy)	Keep only what the law allows
Server watch alerts + daily summary (Telegram)	Know before customers do

---

12. Environment and configuration

Never commit values. Names only:

Backend `.env` — `NODE_ENV`, `PORT`, `APP_ROLE` (`admin` for the admin copy), `APP_TZ_OFFSET`, `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_DATABASE`, `DB_POOL_*`, `JWT_SECRET`, `DATA_ENCRYPTION_KEY`, `CLIENT_URL` / `FRONTEND_URL`, `DASHBOARD_URL`, `REDIS_URL` (optional queue/cache), `SENTRY_DSN`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_CHAT_ID`, `WHATSAPP_APP_ID`, `WHATSAPP_APP_SECRET`, `WHATSAPP_WEBHOOK_VERIFY_TOKEN`, `META_APP_ID`, `META_APP_SECRET`, `VISIT_INGEST_KEY`, `VAPID_*` (web push), `CRON_TOKEN`, `INVOICE_URL_SECRET`.

Per store, in `brand_settings` (Dashboard → Settings) — payment gateway keys + webhook secret, shipping provider keys, hosted checkout keys, WhatsApp phone number id + token + templates, `FB_PIXEL_ID`, Conversions API token, `GA_MEASUREMENT_ID`, GA API secret, `GTM_ID`, `CLARITY_ID`, Google Ads ids/labels, policies.

Storefront (Vercel) — `NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_BRAND_NAME`, `VISIT_INGEST_KEY` (edge → API).

---

13. Deploy

- **Backend**: GitHub Actions `backend-deploy-ftp.yml` — install, run tests, upload over FTP to the main app and the admin copy, touch `tmp/restart.txt` to restart. Tests failing = no deploy.
- **Storefront + Dashboard**: `vercel-deploy-all.yml` calls each project's Vercel deploy hook when its folder changed.
- **Shared storefront files**: edit only in `shared/frontend/`, then `node scripts/sync-frontend-shared.mjs` (CI checks with `--check`).
- Every page carries `<meta name="x-build">` with the commit, so you can see which version is live.
- Database changes are additive (`CREATE TABLE IF NOT EXISTS`, `ADD COLUMN` that ignores "duplicate column"). Never run destructive SQL on production; take a backup first.

---

14. Launch checklist

- ☐ Domain, SSL, `www` → apex (or reverse) redirect; canonical URLs
- ☐ Store row in `brands`; settings filled; secrets encrypted
- ☐ Products with variants, prices, stock, photos, SEO titles/descriptions
- ☐ Policies: privacy, terms, shipping, returns/refunds, contact details
- ☐ Test orders: COD, online paid, online failed, coupon, cancelled, refunded, returned
- ☐ Courier booking and tracking webhook verified with a real shipment
- ☐ WhatsApp templates approved; OTP login works
- ☐ Meta: pixel + Conversions API, domain verified, test events deduplicated, test code removed, catalogue feed connected
- ☐ GA4: Measurement ID + API secret, **Check Google Analytics** passes, purchase shows in DebugView
- ☐ Ads carry UTM parameters; Traffic report Visit check shows "tagged as paid ads"
- ☐ Lighthouse mobile ≥ 85 on home and a product page
- ☐ Cron jobs running (Dashboard → System); Telegram alerts arrive
- ☐ Backups scheduled; Sentry receiving errors
- ☐ Consent banner and consent log; data export/erasure work

---

15. Mistakes we already made (don't repeat)

1. **Falling back to the first store** when the store header was missing — events and orders went to the wrong store.
2. **Trusting the browser's price** — always re-price from the database.
3. **Side effects inside the order transaction** — a rollback still sent WhatsApp/pixel events.
4. **Counting every page load as a visit** — Meta's ad review, link preloads and bots inflated "Facebook" visits.
5. **Ads without UTM tags** — GA showed ad visitors as Direct; ROAS could not be matched.
6. **Loading analytics late for ad visitors** — short visits were never counted by GA/Meta.
7. **Uploading the whole catalogue to the hosted checkout on every tap** — the checkout took seconds to open.
8. **Report days in UTC** — orders between 00:00 and 05:30 IST landed on the wrong day.

9. **A test event code left on the Conversions API** — real events went to Test Events only.
10. **Mixing old and new tracking data** after a fix — set a clean “data from” date instead.